

TECHNICAL REFERENCE GUIDE

Qwik Guide: Data Preprocessing

Scaling · Binarizing · Normalizing · Label & One-Hot Encoding

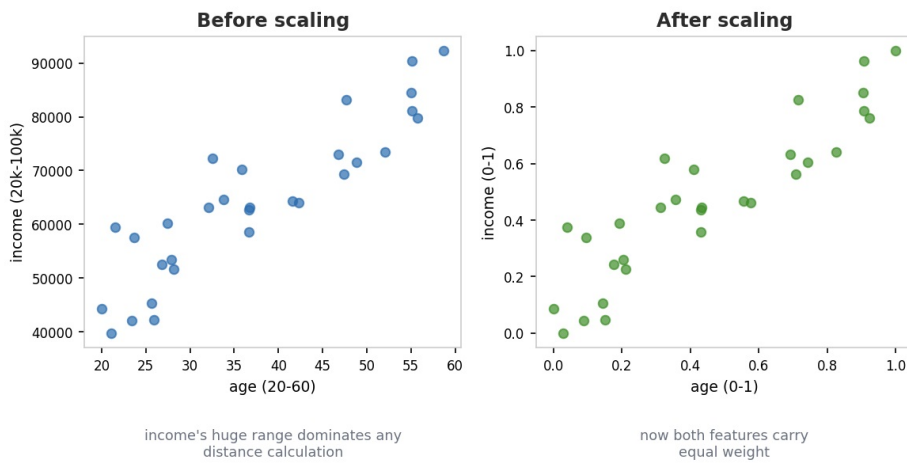
A beginner-friendly companion for the AI Foundations Series.

Plain-language definitions, worked scikit-learn examples, and diagrams — a partner volume to the Applying Machine Learning guide.

Models don't learn from raw data — they learn from data that's been put in shape. Numbers on wildly different scales need scaling; categories like "red" or "Sedan" need turning into numbers. This guide walks through the scikit-learn and Pandas tools that do this, defining every term and showing the exact transformation each one performs.

Why Preprocess Data?

Two things stop raw data from being model-ready. First, numeric features often live on very different **scales** — an age runs 0–100 while an income runs into the hundreds of thousands — and many algorithms let the big-range feature dominate. Second, many columns are **text categories** that a model can't do math on. Preprocessing fixes both.



Left: income's huge range would dominate any distance-based calculation. Right: after scaling, both features contribute equally.

TIP A key idea for every scaler here: you fit on the **training data only** (learning its min, max, mean, etc.), then apply that same transform to new data. scikit-learn's `fit_transform` does both at once for the training set.

1 Exploring a Dataset First

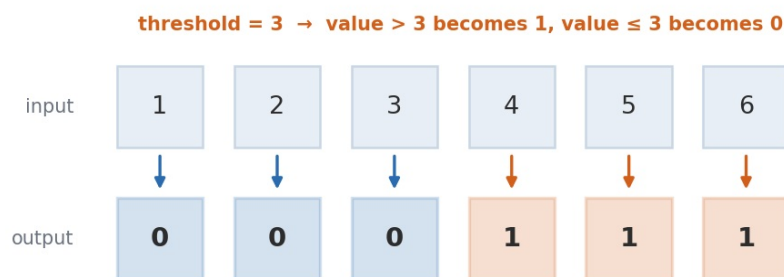
Before transforming anything, load and look at the data. The **Pandas** library reads a CSV into a **DataFrame** — a table of rows and columns — and gives quick tools for exploratory analysis.

- `read_csv()` — loads a CSV file into a `DataFrame`.
- `head()` — shows the first few rows so you can eyeball the columns.
- `describe()` — the summary of the distributions of all numeric fields: count, mean, standard deviation, min, the quartiles, and max, for every numeric column at once.

```
import pandas as pd
df = pd.read_csv("data.csv")
df.head()      # peek at the first rows
df.describe()  # summary stats for every numeric column
```

2 Binarizer — Continuous to Binary

A **Binarizer** turns continuous numbers into just 0s and 1s using a **threshold**: every value *greater than* the threshold becomes **1**, and every value *less than or equal to* it becomes **0**. Useful when you only care whether a measurement crosses some cutoff.



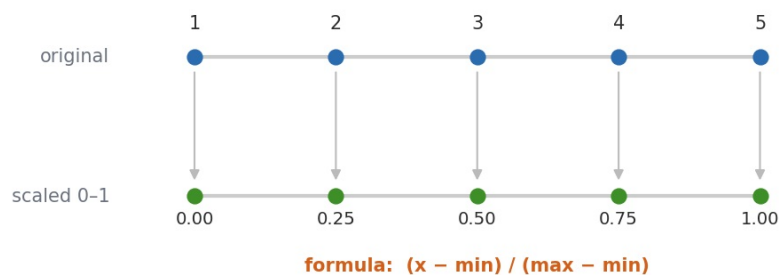
With threshold = 3, the values 1, 2, 3 fall to 0 and 4, 5, 6 rise to 1.

```
from sklearn.preprocessing import Binarizer
Binarizer(threshold=3).fit_transform([[1],[2],[3],[4],[5],[6]])
# → [[0],[0],[0],[1],[1],[1]]
```

3 MinMaxScaler — Squeeze to a Range

The **MinMaxScaler** rescales a column so its smallest value becomes the bottom of a target range and its largest becomes the top — by default **0 to 1**. This is how you get two columns with different ranges onto the same footing.

$$\text{scaled} = (x - \min) / (\max - \min) \times (\text{range size})$$



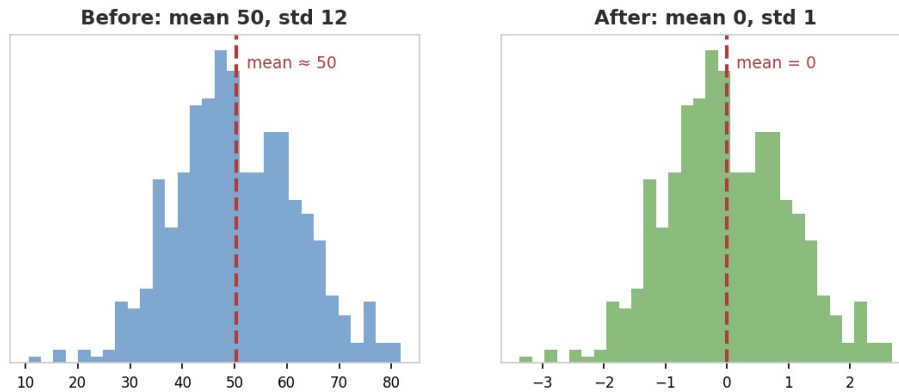
The minimum maps to the bottom of the range, the maximum to the top, and everything else spaces out proportionally in between.

```
from sklearn.preprocessing import MinMaxScaler
MinMaxScaler(feature_range=(0, 5)).fit_transform([[1],[2],[3],[4],[5]])
# → [[0],[1.25],[2.5],[3.75],[5]]
```

4 StandardScaler — Standardization

Standardization transforms an attribute that has a Gaussian (bell-curve) distribution with some mean and spread into a **standard Gaussian** — one centered at **mean 0** with a **standard deviation of 1**. It doesn't squeeze data into a fixed range like MinMax; instead it re-expresses each value as "how many standard deviations from the average."

$$z = (x - \text{mean}) / \text{standard deviation}$$



Standardization slides the distribution so its mean sits at 0 and stretches or squeezes it so its standard deviation is 1 — the shape is preserved.

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaled = scaler.fit_transform(df[["height", "weight"]])
# each column now has mean 0 and standard deviation 1
```

WHEN TO USE IT Standardization is a good default for algorithms that assume roughly centered data or measure distances (many linear models, SVMs, k-NN, PCA). MinMax is handy when you specifically need a bounded range like 0-1.

5 Normalizer — the Row-wise Scaler

The **Normalizer** is the odd one out, and it's the distinction students miss most. Every other scaler in this guide works **down a column** — it looks at all the values of one feature and rescales them together. The Normalizer instead works **across each row**, rescaling every sample on its own so that the row's values have unit length (unit norm).

StandardScaler / MinMaxScaler
work **DOWN** each column

2	1	2
4	3	0
1	1	1
0	5	2

Normalizer works ACROSS
each row (sample) on its own

2	1	2
4	3	0
1	1	1
0	5	2

The difference in one picture: column scalers operate feature by feature (down); the Normalizer operates sample by sample (across).

The distinction to remember

StandardScaler, MinMaxScaler, MaxAbsScaler → operate on **columns** (features). Each feature is rescaled using that feature's own statistics across all rows.

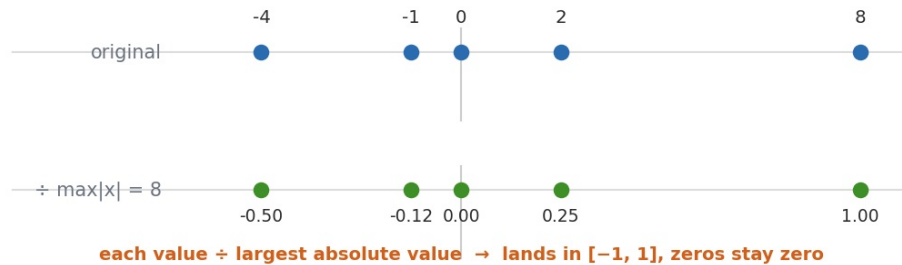
Normalizer → operates on **rows** (samples). Each row is rescaled independently of every other row. It's common when the *direction* of a sample's values matters more than their magnitude — for example text documents represented as word counts.

```
from sklearn.preprocessing import Normalizer
Normalizer().fit_transform(df[continuous_cols])
# each ROW is scaled to unit norm, independently
```

6 MaxAbsScaler — Proportion of the Maximum

The **MaxAbsScaler** divides every value in a feature by that feature's **maximum absolute value**. The result expresses each value as a *proportion* of the biggest one, landing in the range **[−1, 1]**. It scales without shifting or centering the data, so a zero stays a zero — which keeps sparse data sparse.

$$\text{scaled} = x / \max(|x|)$$



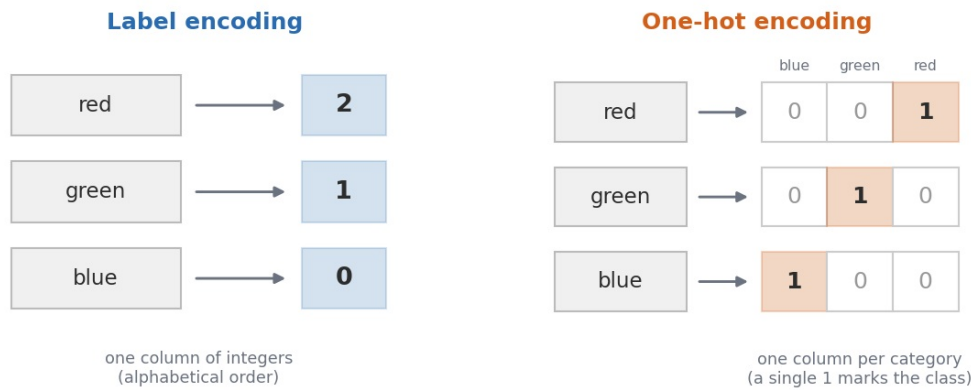
Dividing by the largest absolute value (8 here) maps everything into [−1, 1] while leaving zero fixed in place.

```
from sklearn.preprocessing import MaxAbsScaler
MaxAbsScaler().fit_transform(df[["sales"]])
# each product's sales as a fraction of the top product's
```

THREE SCALERS, SIDE BY SIDE **MinMax** stretches min→max into a chosen range (default 0–1). **Standard** re-centers to mean 0, std 1. **MaxAbs** divides by the largest magnitude into [−1, 1] without shifting. All three work column by column.

7 Encoding Categories as Numbers

Models need numbers, but many columns hold text categories ("red", "Sedan"). Two techniques convert them — and choosing the wrong one can quietly mislead your model.



Label encoding produces one integer column; one-hot encoding produces one column per category with a single 1 marking the class.

Label encoding

Label encoding assigns each distinct category an integer, **in alphabetical order**: the first category alphabetically becomes 0, the next 1, and so on.

THE LIMITATION The integers imply an *order and spacing* that usually isn't real — encoding colors as 0, 1, 2 suggests $\text{green} < \text{red} < \text{yellow}$ and that the gaps are equal. That's fine for a **target** column, but on **input features** it can fool the model into reading meaning into the numbers. Prefer one-hot for unordered input categories.

One-hot encoding

One-hot encoding makes a new column for each category and marks the right one with a 1 (the rest 0). No false ordering is implied — every category is equally distant from every other. Pandas' `get_dummies()` does this in one call.

REMEMBER THE ORDER Both encoders order categories **alphabetically**. For one-hot, the "1" sits in the position of that category's alphabetical rank — so among [Hatchback, Sedan, SUV], sorted to Hatchback, SUV, Sedan, "Sedan" is [0, 0, 1].

★ Concept Check

Worked examples of the exact points students trip on. Read each as "the question, then the answer."

Label-encode fall, winter, spring, summer — what do they become (in that order)?
0, 3, 1, 2. Sorted alphabetically: fall, spring, summer, winter = 0,1,2,3. Mapped back to the original order → 0, 3, 1, 2.

What does MaxAbsScaler do?
Scales each value by its feature's **maximum absolute value**, landing in $[-1, 1]$ without shifting.

MinMaxScaler(feature_range=(0,5)) on 1-6?
[0, 1.25, 2.5, 3.75, 5]. $(x-1)/(5-1) \times 5$.

One-hot value of "yellow" in red, orange, green, yellow?
[0, 0, 0, 1]. Sorted alphabetically (green, orange, red, yellow), yellow is last.

How does the Normalizer differ from other scalers?
It works on **each row (sample) independently**, not down each column.

Which Pandas function summarizes all numeric fields?
describe() — count, mean, std, min, quartiles, max.

Standardization gives what mean and standard deviation?
Mean 0, standard deviation 1 — the standard Gaussian.

★ Quick Reference

Tool	What it does	Result
<code>describe()</code>	Summarizes every numeric column	count, mean, std, min, quartiles, max
Binarizer	Threshold: > threshold → 1, else 0	0s and 1s
MinMaxScaler	Stretch min→max into a range (column)	Default 0–1
StandardScaler	Center & rescale a column (standardize)	Mean 0, std 1
MaxAbsScaler	Divide a column by its max absolute value	[−1, 1], zeros preserved
Normalizer	Rescale each row to unit norm	Per-sample, not per-column
LabelEncoder	Category → integer (alphabetical)	One integer column; implies order
<code>get_dummies</code>	One-hot: a column per category	A single 1 per row; no false order

Column scalers vs. the Normalizer

	Column scalers (Standard / MinMax / MaxAbs)	Normalizer
Operates on	Each feature (column), across all rows	Each sample (row), on its own
Uses	That column's min / max / mean / std	That row's own values
Typical goal	Put features on a comparable scale	Make each sample's direction comparable

Label vs. one-hot encoding

	Label encoding	One-hot encoding
Output	One integer column	One column per category
Implies order?	Yes ($0 < 1 < 2$)	No — all categories equal
Best for	Target column, or ordered categories	Unordered input features